

Argus Security Analysis Report

Repository: https://github.com/arnav-sutraway/Vuln_test

Total Findings: 15

1. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 88.0%

Location: admin.py, line 48

Evidence: query = f"SELECT * FROM audit_logs WHERE action LIKE '%{search_term}%'"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

2. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 11

Evidence: query = "SELECT * FROM users WHERE username = " + username + ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

3. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 60

Evidence: query = f"SELECT * FROM products ORDER BY {sort_column}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

4. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services\reports.py, line 21

Evidence: query = f"SELECT * FROM {table_name}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

5. Debug mode enabled

Vulnerability Type: Insecure Configuration

Severity: Medium

Confidence: 80.0%

Location: app.py, line 72

Evidence: app.run(debug=True)

Description: Debug mode is on. It can expose stack traces, source, and an interactive console.

Recommendation: Turn debug mode off outside local development.

6. Hardcoded credential

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: auth.py, line 9

Evidence: API_KEY = "ARGUS_FAKE_API_KEY_123456789"

Description: A credential is written directly in the file. Anyone with the source can reuse it.

Recommendation: Load the value from the environment or a secret manager, and rotate the exposed credential.

7. Hardcoded credential

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: auth.py, line 13

Evidence: DATABASE_PASSWORD = "ARGUS_TEST_DATABASE_PASSWORD"

Description: A credential is written directly in the file. Anyone with the source can reuse it.

Recommendation: Load the value from the environment or a secret manager, and rotate the exposed credential.

8. Hardcoded credential

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: auth.py, line 39

Evidence: JWT_SECRET = "ARGUS_FAKE_JWT_SIGNING_SECRET"

Description: A credential is written directly in the file. Anyone with the source can reuse it.

Recommendation: Load the value from the environment or a secret manager, and rotate the exposed credential.

9. Debug mode enabled

Vulnerability Type: Insecure Configuration

Severity: Medium

Confidence: 80.0%

Location: challenges\broken_access_control.py, line 22

Evidence: app.run(debug=True)

Description: Debug mode is on. It can expose stack traces, source, and an interactive console.

Recommendation: Turn debug mode off outside local development.

10. Embedded private key or cloud key

Vulnerability Type: Hardcoded Secret

Severity: Critical

Confidence: 95.0%

Location: config.py, line 10

Evidence: AWS_ACCESS_KEY_ID = "AKIAARGUSFAKEACCESSKEY"

Description: The file contains a private key or a live provider key.

Recommendation: Remove the key from the repository, rotate it, and store the replacement outside source control.

11. Hardcoded credential

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: config.py, line 16

Evidence: DATABASE_PASSWORD = "argus_fake_postgres_password"

Description: A credential is written directly in the file. Anyone with the source can reuse it.

Recommendation: Load the value from the environment or a secret manager, and rotate the exposed credential.

12. Debug mode enabled

Vulnerability Type: Insecure Configuration

Severity: Medium

Confidence: 80.0%

Location: config.py, line 29

Evidence: DEBUG = True

Description: Debug mode is on. It can expose stack traces, source, and an interactive console.

Recommendation: Turn debug mode off outside local development.

13. Unsafe HTML injection

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 84.0%

Location: templates\admin.html, line 17

Evidence: {{ comment | safe }}

Description: Untrusted content is written into HTML. If any part of this value comes from a request, an attacker can run script in the user's browser.

Recommendation: Insert text instead of HTML, or sanitize with a dedicated HTML sanitizer before rendering.

14. Unsafe HTML injection

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 84.0%

Location: templates\profile.html, line 15

Evidence: {{ bio | safe }}

Description: Untrusted content is written into HTML. If any part of this value comes from a request, an attacker can run script in the user's browser.

Recommendation: Insert text instead of HTML, or sanitize with a dedicated HTML sanitizer before rendering.

15. Unsafe HTML injection

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 84.0%

Location: templates\search.html, line 15

Evidence: {{ query | safe }}

Description: Untrusted content is written into HTML. If any part of this value comes from a request, an attacker can run script in the user's browser.

Recommendation: Insert text instead of HTML, or sanitize with a dedicated HTML sanitizer before rendering.